MATLAB Code Collection for Numerical Methods

Numerical Methods Course

September 16, 2025

Contents

1 Taylor Series Implementation **3**
- 1.1 taylor_series.m . . . 3
- 1.2 run_taylor_series.m . . . 3
- 1.3 diffex.m . . . 3
- 1.4 run_diffex.m . . . 4
- 1.5 practice_problems.m . . . 4
- 1.6 exercise_1.m . . . 5

2 Root Finding Methods **5**
- 2.1 bisection_method.m . . . 5
- 2.2 false_position.m . . . 6
- 2.3 run_bisection_example.m . . . 7
- 2.4 run_false_position_example.m . . . 7
- 2.5 solve_practice_problem.m . . . 8
- 2.6 compare_methods.m . . . 9

3 Newton-Raphson and Secant Methods **9**
- 3.1 newton_raphson.m . . . 9
- 3.2 secant_method.m . . . 10
- 3.3 compare_methods_newton.m . . . 11

4 LU Decomposition **12**
- 4.1 lu_decomposition_solve.m . . . 12
- 4.2 practice_problem_1.m . . . 13
- 4.3 practice_problem_2.m . . . 13
- 4.4 test_examples.m . . . 14

5 Golden-Section Search **15**
- 5.1 golden_section_search.m . . . 15
- 5.2 run_golden_section_example.m . . . 16
- 5.3 beam_deflection.m . . . 16
- 5.4 projectile_altitude.m . . . 17

6 ODE Solvers **18**
- 6.1 euler_method.m . . . 18
- 6.2 runge_kutta4.m . . . 18
- 6.3 heun_method.m . . . 19
- 6.4 run_ode_examples.m . . . 19
- 6.5 compare_ode_methods.m . . . 21

7 Numerical Integration **22**
7.1 trapezoidal_rule.m 22
7.2 simpsons_13_rule.m 22
7.3 simpsons_38_rule.m 23
7.4 run_integration_examples.m 24

1 Taylor Series Implementation

1.1 taylor_series.m

```
function [taylor_approx] = taylor_series(func, degree, point, x)
    % Initialize the Taylor series approximation
    taylor_approx = 0;

    % Iterate through each term in the Taylor series
    for n = 0:degree
        % Calculate the nth derivative of the function at the point
        nth_derivative = diff(func, n);

        % Evaluate the nth derivative at the point
        nth_derivative_value = double(subs(nth_derivative, point));

        % Calculate the nth term of the Taylor series
        nth_term = (nth_derivative_value / factorial(n)) * (x - point)^n;

        % Add the nth term to the Taylor series approximation
        taylor_approx = taylor_approx + nth_term;
    end
end
```

1.2 run_taylor_series.m

```
% Script to call taylor_series for sin(x)
syms x;
func = sin(x);
degree = 3;
point = 0; % the value of a in h=(x-a)

taylor_approx = taylor_series(func, degree, point, x);

% Plot the original function and its Taylor series approximation
figure;
hold on;
fplot(func, [-pi, pi], 'LineWidth', 2);
fplot(taylor_approx, [-pi, pi], '--r', 'LineWidth', 1.5);
legend('Original Function', 'Taylor Series Approximation');
xlabel('x');
ylabel('y');
title('Taylor Series Approximation of sin(x)');

grid on;
zoom on;
hold off;

% Alternative using built-in taylor function
% Note: x_values needs to be defined for this to work
% A3 = double(subs(taylor(func, x, 'ExpansionPoint', point, 'Order', 3),
    x_values))
```

1.3 diffex.m

```
function diffex(func, dfunc, x, n)
    format long
    dftrue = dfunc(x);
    h = 1;
    H(1) = h;
```

```
    D(1) = (func(x+h) - func(x-h)) / (2*h);
    E(1) = abs(dftrue - D(1));

    for i = 2:n
        h = h/10;
        H(i) = h;
        D(i) = (func(x+h) - func(x-h)) / (2*h);
        E(i) = abs(dftrue - D(i));
    end

    L = [H' D' E']';
    fprintf(' step size finite difference true error\n');
    fprintf('%14.10f %16.14f %16.13f\n', L);
    loglog(H, E), xlabel('Step Size'), ylabel('Error')
    title('Plot of Error Versus Step Size')
    format short
end
```

1.4 run_diffex.m

```
% Call diffex function
ff = @(x) -0.1*x^4 - 0.15*x^3 - 0.5*x^2 - 0.25*x + 1.2;
df = @(x) -0.4*x^3 - 0.45*x^2 - x - 0.25;
diffex(ff, df, 0.5, 11);
```

1.5 practice_problems.m

```
% Practice Problems from the document

% 1. Taylor series for sin(x) up to 6th, 8th, and 10th order
syms x;
figure;
hold on;
fplot(sin(x), [-pi, pi], 'LineWidth', 2);

% 6th order
taylor_6 = taylor(sin(x), x, 'Order', 7); % Order is n+1
fplot(taylor_6, [-pi, pi], '--', 'LineWidth', 1.5);

% 8th order
taylor_8 = taylor(sin(x), x, 'Order', 9);
fplot(taylor_8, [-pi, pi], '--', 'LineWidth', 1.5);

% 10th order
taylor_10 = taylor(sin(x), x, 'Order', 11);
fplot(taylor_10, [-pi, pi], '--', 'LineWidth', 1.5);

legend('sin(x)', '6th Order', '8th Order', '10th Order');
xlabel('x');
ylabel('y');
title('Taylor Series Approximations of sin(x)');
grid on;
hold off;

% 2. cos(x) up to 8th order
figure;
hold on;
fplot(cos(x), [-pi, pi], 'LineWidth', 2);
taylor_cos = taylor(cos(x), x, 'Order', 9);
fplot(taylor_cos, [-pi, pi], '--r', 'LineWidth', 1.5);
legend('cos(x)', '8th Order Approximation');
```

```
xlabel('x');
ylabel('y');
title('Taylor Series Approximation of cos(x)');
grid on;
hold off;

% 3. 1/(1+x) up to 10th order
figure;
hold on;
fplot(1/(1+x), [-0.5, 2], 'LineWidth', 2);
taylor_frac = taylor(1/(1+x), x, 'Order', 11);
fplot(taylor_frac, [-0.5, 2], '--r', 'LineWidth', 1.5);
legend('1/(1+x)', '10th Order Approximation');
xlabel('x');
ylabel('y');
title('Taylor Series Approximation of 1/(1+x)');
grid on;
hold off;
```

1.6 exercise_1.m

```
% Exercise 1: Compute first 4 terms of Taylor series

% a) f(x) = 1/x about a = -1
syms x;
f_a = 1/x;
a = -1;
taylor_a = taylor(f_a, x, a, 'Order', 4);
disp('Taylor series for 1/x about a = -1:');
disp(taylor_a);

% b) f(x) = -0.1x^4 - 0.15x^3 - 0.5x^2 - 0.25x + 1.2 about a = 0
f_b = -0.1*x^4 - 0.15*x^3 - 0.5*x^2 - 0.25*x + 1.2;
a_b = 0;
taylor_b = taylor(f_b, x, a_b, 'Order', 4);
disp('Taylor series for polynomial about a = 0:');
disp(taylor_b);

% Evaluate at x = 1
value_at_1 = double(subs(taylor_b, x, 1));
disp('Value at x = 1:');
disp(value_at_1);
```

2 Root Finding Methods

2.1 bisection_method.m

```
function [iteration, XL, XU, rootValue, err] = bisection_method(func, xl, xu,
    err_limit, max_iter)
    % Check if the function has different signs at the interval boundaries
    if sign(func(xl)) == sign(func(xu))
        error('Function has the same sign at interval boundaries. Bisection
    method cannot proceed.');
    end

    % Initialize iteration counter and error
    iter = 1;
    iteration(1) = 0;
    rootValue(1) = 0;
    err(1) = 100;
```

```
    XL(1) = xl;
    XU(1) = xu;
    root = 0;

    % Bisection method iteration
    while err(iter) > err_limit && iter < max_iter
        % Midpoint of the interval (corrected from original)
        c = (xl + xu) / 2;

        % Evaluate the function at the midpoint
        fc = func(c);

        % Update the interval based on the signs of function evaluations
        if sign(func(xl)) * sign(fc) < 0
            xu = c;
        else
            xl = c;
        end

        % Update the error and iteration count
        iter = iter + 1;
        rootValue(iter) = c;
        err(iter) = abs(c - root)/c * 100;
        iteration(iter) = iter - 1;
        XL(iter) = xl;
        XU(iter) = xu;
        % update the root approximation
        root = c;
    end
end
```

2.2 false_position.m

```
function [iteration, XL, XU, rootValue, err] = false_position(func, xl, xu,
    err_limit, max_iter)
    % Check if the function has different signs at the interval boundaries
    if sign(func(xl)) == sign(func(xu))
        error('Function has the same sign at interval boundaries. False
    position method cannot proceed.');
    end

    % Initialize iteration counter and error
    iter = 1;
    iteration(1) = 0;
    rootValue(1) = 0;
    err(1) = 100;
    root = 0;
    XL(1) = xl;
    XU(1) = xu;

    % False position method iteration
    while err(iter) > err_limit && iter < max_iter
        % Calculate the new estimate using false position formula
        xr = xu - (func(xu) * (xl - xu)) / (func(xl) - func(xu));

        % Evaluate the function at the new estimate
        fr = func(xr);

        % Update the interval based on the signs of function evaluations
        if sign(func(xl)) * sign(fr) < 0
            xu = xr;
        else
```

```
            xl = xr;
        end

        % Update the error and iteration count
        iter = iter + 1;
        rootValue(iter) = xr;
        err(iter) = abs(xr - root)/xr * 100;
        iteration(iter) = iter - 1;
        XL(iter) = xl;
        XU(iter) = xu;

        % update the root approximation
        root = xr;
    end
end
```

2.3 run_bisection_example.m

```
% Script to run the bisection method for the practice problem
% Define the function
func = @(x) sin(10*x) + cos(3*x);

% Set the interval and parameters
xl = 12;
xu = 16;
err_limit = 0.5; % 0.5%
max_iter = 100;

% Run the bisection method
[iteration, XL, XU, rootValue, err] = bisection_method(func, xl, xu, err_limit,
     max_iter);

% Display results
fprintf('Iteration\tXL\t\tXU\t\tRoot\t\tError(%%)\n');
for i = 1:length(iteration)
    fprintf('%d\t\t%.6f\t%.6f\t%.6f\t%.6f\n', ...
            iteration(i), XL(i), XU(i), rootValue(i), err(i));
end

% Plot the errors
figure;
plot(iteration(2:end), err(2:end), 'b-o', 'LineWidth', 2);
xlabel('Iteration');
ylabel('Percent Relative Error');
title('Error vs. Iteration for Bisection Method');
grid on;
```

2.4 run_false_position_example.m

```
% Script to run the false position method
% Define the function
func = @(x) sin(10*x) + cos(3*x);

% Set the interval and parameters
xl = 12;
xu = 16;
err_limit = 0.5; % 0.5%
max_iter = 100;

% Run the false position method
```

```
[iteration, XL, XU, rootValue, err] = false_position(func, xl, xu, err_limit,
    max_iter);

% Display results
fprintf('Iteration\tXL\t\tXU\t\tRoot\t\tError(%%)\n');
for i = 1:length(iteration)
    fprintf('%d\t\t%.6f\t%.6f\t%.6f\t%.6f\n', ...
            iteration(i), XL(i), XU(i), rootValue(i), err(i));
end

% Plot the errors
figure;
plot(iteration(2:end), err(2:end), 'r-o', 'LineWidth', 2);
xlabel('Iteration');
ylabel('Percent Relative Error');
title('Error vs. Iteration for False Position Method');
grid on;
```

2.5 solve_practice_problem.m

```
% Practice Problem Solution
% Solve f(x) = sin(10x) + cos(3x) in the interval [12,16] using bisection
    method

% Define the function
func = @(x) sin(10*x) + cos(3*x);

% Set parameters
xl = 12;
xu = 16;
err_limit = 0.5; % 0.5%
max_iter = 100;

% Apply bisection method
[iteration, XL, XU, rootValue, err] = bisection_method(func, xl, xu, err_limit,
     max_iter);

% Display results
fprintf('Bisection Method Results for f(x) = sin(10x) + cos(3x)\n');
fprintf('Iteration\tXL\t\tXU\t\tRoot\t\tError(%%)\n');
for i = 1:length(iteration)
    fprintf('%d\t\t%.6f\t%.6f\t%.6f\t%.6f\n', ...
            iteration(i), XL(i), XU(i), rootValue(i), err(i));
end

% Plot the function to visualize roots
figure;
x = linspace(12, 16, 1000);
y = func(x);
plot(x, y, 'b-', 'LineWidth', 2);
hold on;
plot(rootValue(end), func(rootValue(end)), 'ro', 'MarkerSize', 10, '
    MarkerFaceColor', 'r');
xlabel('x');
ylabel('f(x)');
title('Function: f(x) = sin(10x) + cos(3x)');
grid on;
legend('Function', 'Estimated Root');

% Plot the error convergence
figure;
semilogy(iteration(2:end), err(2:end), 'b-o', 'LineWidth', 2);
```

```
xlabel('Iteration');
ylabel('Percent Relative Error (%)');
title('Error Convergence in Bisection Method');
grid on;

fprintf('\nFinal root approximation: %.6f\n', rootValue(end));
fprintf('Final error: %.6f%%\n', err(end));
fprintf('Function value at root: %.6f\n', func(rootValue(end)));
```

2.6 compare_methods.m

```
% Compare Bisection and False Position methods
func = @(x) sin(10*x) + cos(3*x);
xl = 12;
xu = 16;
err_limit = 0.5;
max_iter = 100;

% Run both methods
[iter_b, XL_b, XU_b, root_b, err_b] = bisection_method(func, xl, xu, err_limit,
    max_iter);
[iter_f, XL_f, XU_f, root_f, err_f] = false_position(func, xl, xu, err_limit,
    max_iter);

% Plot error comparison
figure;
semilogy(iter_b(2:end), err_b(2:end), 'b-o', 'LineWidth', 2);
hold on;
semilogy(iter_f(2:end), err_f(2:end), 'r-*', 'LineWidth', 2);
xlabel('Iteration');
ylabel('Percent Relative Error (%)');
title('Error Comparison: Bisection vs False Position');
legend('Bisection', 'False Position');
grid on;

% Display comparison results
fprintf('Method\t\tFinal Root\tIterations\tFinal Error\n');
fprintf('Bisection\t%.6f\t%d\t\t%.6f%%\n', root_b(end), length(iter_b)-1, err_b
    (end));
fprintf('False Position\t%.6f\t%d\t\t%.6f%%\n', root_f(end), length(iter_f)-1,
    err_f(end));
```

3 Newton-Raphson and Secant Methods

3.1 newton_raphson.m

```
function [iteration, rootValue, err] = newton_raphson(func, dfunc, x0,
    err_limit, max_iter)
    % Initialize arrays for storing iteration, root value, and error
    iter = 1;
    iteration(1) = 0;
    rootValue(1) = x0;
    err(1) = 100;

    % Newton-Raphson method iteration
    x = x0;
    while err(iter) > err_limit && iter < max_iter
        % Compute the function value and its derivative at the current point
        fx = func(x);
        dfx = dfunc(x);
```

```

        % Avoid division by zero
        if dfx == 0
            error('Derivative is zero. Cannot continue Newton-Raphson method.');
        end

        % Compute the next approximation of the root using Newton's method
        x_new = x - fx / dfx;

        % Store the current values
        iter = iter + 1;
        rootValue(iter) = x_new;
        iteration(iter) = iter - 1;
        err(iter) = abs(x_new - x) / x_new * 100;

        % Update the current point for the next iteration
        x = x_new;
    end
end
```

3.2 secant_method.m

```
function [iteration, XL, XU, rootValue, err] = secant_method(func, x0, x1, err_limit, max_iter)
    % Initialize arrays for storing iteration, root value, and error
    iter = 1;
    iteration(1) = 0;
    rootValue(1) = 0;
    err(1) = 100;
    XL(1) = x0;
    XU(1) = x1;

    % Secant method iteration
    while err(iter) > err_limit && iter < max_iter
        % Evaluate the function at current points
        f0 = func(x0);
        f1 = func(x1);

        % Avoid division by zero
        if (f1 - f0) == 0
            error('Difference in function values is zero. Cannot continue Secant method.');
        end

        % Compute the next approximation of the root using the secant method
        x_new = x1 - f1 * (x1 - x0) / (f1 - f0);

        % Store the current value
        iter = iter + 1;
        rootValue(iter) = x_new;
        iteration(iter) = iter - 1;
        err(iter) = abs(x_new - x1) / x_new * 100;
        XL(iter) = x0;
        XU(iter) = x1;

        % Update the current points for the next iteration
        x0 = x1;
        x1 = x_new;
    end
end
```

3.3 compare_methods_newton.m

```
% Practice Problem: Compare methods for f(x) = e^(-x) - x
% True root is 0.56714329

% Define the function and its derivative
func = @(x) exp(-x) - x;
dfunc = @(x) -exp(-x) - 1;

% Set parameters
true_root = 0.56714329;
err_limit = 0.0001; % 0.01% error limit
max_iter = 10;

% Newton-Raphson method
x0_newton = 0.5; % Initial guess
[iter_n, root_n, err_n] = newton_raphson(func, dfunc, x0_newton, err_limit,
    max_iter);

% Secant method
x0_secant = 0.5;
x1_secant = 0.6;
[iter_s, XL_s, XU_s, root_s, err_s] = secant_method(func, x0_secant, x1_secant,
     err_limit, max_iter);

% Calculate true errors
true_err_n = abs(root_n - true_root) / true_root * 100;
true_err_s = abs(root_s - true_root) / true_root * 100;

% Display results
fprintf('Newton-Raphson Method:\n');
fprintf('Iteration\tRoot\t\tTrue Error(%%)\n');
for i = 1:length(iter_n)
    fprintf('%d\t\t%.8f\t%.8f\n', iter_n(i), root_n(i), true_err_n(i));
end

fprintf('\nSecant Method:\n');
fprintf('Iteration\tRoot\t\tTrue Error(%%)\n');
for i = 1:length(iter_s)
    fprintf('%d\t\t%.8f\t%.8f\n', iter_s(i), root_s(i), true_err_s(i));
end

% Plot results
figure;

% Newton-Raphson convergence
subplot(2, 1, 1);
semilogy(iter_n, true_err_n, 'b-o', 'LineWidth', 2);
xlabel('Iteration');
ylabel('True Percent Relative Error (%)');
title('Newton-Raphson Method Convergence');
grid on;

% Secant method convergence
subplot(2, 1, 2);
semilogy(iter_s, true_err_s, 'r-*', 'LineWidth', 2);
xlabel('Iteration');
ylabel('True Percent Relative Error (%)');
title('Secant Method Convergence');
grid on;

% Display final results
fprintf('\nFinal Results:\n');
```

```
fprintf('Method\t\tFinal Root\tTrue Error(%%)\tIterations\n');
fprintf('Newton-Raphson\t%.8f\t%.8f\t%d\n', root_n(end), true_err_n(end),
    length(iter_n)-1);
fprintf('Secant\t\t%.8f\t%.8f\t%d\n', root_s(end), true_err_s(end), length(
    iter_s)-1);
```

4 LU Decomposition

4.1 lu_decomposition_solve.m

```
function [X, L, U, P] = lu_decomposition_solve(A, B)
    % Get the size of A
    [row, column] = size(A);
    n = row;

    % Initialize U, L, and P (permutation matrix)
    U = A;
    L = eye(n);
    P = eye(n);

    % Initialize permutation vector
    p = (1:n)';

    % Modifying Matrix A for Pivot element before U and L matrix
    for j = 1:column-1
        % Find the pivot row
        [~, max_row] = max(abs(U(j:n, j)));
        max_row = max_row + j - 1;

        if j ~= max_row
            % Swap rows in U
            temp = U(j,:);
            U(j,:) = U(max_row,:);
            U(max_row,:) = temp;

            % Swap rows in L for the columns already computed
            if j > 1
                temp = L(j, 1:j-1);
                L(j, 1:j-1) = L(max_row, 1:j-1);
                L(max_row, 1:j-1) = temp;
            end

            % Swap rows in P
            temp = P(j,:);
            P(j,:) = P(max_row,:);
            P(max_row,:) = temp;

            % Swap rows in permutation vector
            temp = p(j);
            p(j) = p(max_row);
            p(max_row) = temp;

            % Swap rows in B
            temp = B(j);
            B(j) = B(max_row);
            B(max_row) = temp;
        end

        % Compute the multipliers and update U and L
        for i = j+1:row
            factor = U(i, j) / U(j, j);
```

```
            L(i, j) = factor;
            U(i, j:column) = U(i, j:column) - factor * U(j, j:column);
        end
    end

    % Forward substitution: L * Y = P * B
    Y = zeros(n, 1);
    for i = 1:n
        Y(i) = B(i) - L(i, 1:i-1) * Y(1:i-1);
    end

    % Backward substitution: U * X = Y
    X = zeros(n, 1);
    for i = n:-1:1
        X(i) = (Y(i) - U(i, i+1:n) * X(i+1:n)) / U(i, i);
    end
end
```

4.2 practice_problem_1.m

```
% Practice Problem 1: Reactor Concentrations

% Define the matrix A and vector B
A = [15, -3, -1;
     -3, 18, -6;
     -4, -1, 12];
B = [3800; 1200; 2350];

% Solve using LU decomposition
[c, L, U, P] = lu_decomposition_solve(A, B);

fprintf('Original problem:\n');
fprintf('c1 = %.2f g/m^3\n', c(1));
fprintf('c2 = %.2f g/m^3\n', c(2));
fprintf('c3 = %.2f g/m^3\n', c(3));

% Part 2: Reduced mass input
B_new = [3800-500; 1200-250; 2350]; % [3300; 950; 2350]

[c_new, ~, ~, ~] = lu_decomposition_solve(A, B_new);

fprintf('\nWith reduced mass input:\n');
fprintf('c1 = %.2f g/m^3\n', c_new(1));
fprintf('c2 = %.2f g/m^3\n', c_new(2));
fprintf('c3 = %.2f g/m^3\n', c_new(3));

fprintf('\nReduction in c3: %.2f g/m^3\n', c(3) - c_new(3));
```

4.3 practice_problem_2.m

```
% Practice Problem 2: Reactor Mass Balance

% Based on the figure, we need to set up mass balance equations
% Assuming the following flow rates and concentrations:
% Let c1, c2, c3 be concentrations in reactors 1, 2, 3
% Mass balance equations would typically be of the form:
% (Flow in) = (Flow out)

% Without the exact figure, we'll assume a common configuration:
% Let Q be flow rates between reactors
% A typical set of equations might be:
```

```
% (Q01 + Q31)*c1 - Q13*c3 = Q01*c01
% (Q12 + Q32)*c2 - Q21*c1 - Q23*c3 = 0
% (Q13 + Q23)*c3 - Q31*c1 - Q32*c2 = 0

% For demonstration, let's assume specific values:
Q01 = 100; Q13 = 50; Q31 = 30;
Q12 = 40; Q21 = 20; Q23 = 60; Q32 = 25;
c01 = 10; % Input concentration

% Set up the equations
A = [Q01+Q31, 0, -Q13;
     -Q21, Q12+Q32, -Q23;
     -Q31, -Q32, Q13+Q23];
B = [Q01*c01; 0; 0];

% Solve using LU decomposition
[c, L, U, P] = lu_decomposition_solve(A, B);

fprintf('Reactor concentrations:\n');
fprintf('c1 = %.2f mg/m^3\n', c(1));
fprintf('c2 = %.2f mg/m^3\n', c(2));
fprintf('c3 = %.2f mg/m^3\n', c(3));
```

4.4 test_examples.m

```
% Test the LU decomposition with various examples from the document

% Example 1: Parachutist problem
fprintf('Parachutist problem:\n');
A1 = [70, 1, 0; 60, -1, 1; 40, 0, -1];
B1 = [636.7; 518.6; 307.4];
[x1, L1, U1, P1] = lu_decomposition_solve(A1, B1);
disp('Solution:');
disp(x1);

% Example 2: Truss problem
fprintf('\nTruss problem:\n');
A2 = [0.866, 0, -0.5, 0, 0, 0;
      0.5, 0, 0.866, 0, 0, 0;
      0, 1, 0.5, 0, 0, 0;
      -0.866, -1, 0, -1, 0, 0;
      -0.5, 0, 0, 0, -1, 0;
      0, 0, -0.866, 0, 0, -1];
B2 = [0; -1000; 0; 0; 0; 0];
[x2, L2, U2, P2] = lu_decomposition_solve(A2, B2);
disp('Solution:');
disp(x2);

% Example 3: Circuit problem
fprintf('\nCircuit problem:\n');
A3 = [1, 1, 1, 0, 0, 0;
      0, -1, 0, 1, -1, 0;
      0, 0, -1, 0, 0, 1;
      0, 0, 0, 0, 1, -1;
      0, 10, -10, 0, -15, -5;
      5, -10, 0, -20, 0, 0];
B3 = [0; 0; 0; 0; 200; 0];
[x3, L3, U3, P3] = lu_decomposition_solve(A3, B3);
disp('Solution:');
disp(x3);
```

5 Golden-Section Search

5.1 golden_section_search.m

```
function [x_opt, f_opt, history] = golden_section_search(func, xL, xU, err_limit, max_iter)
    % Golden-section search for maximum of a function
    % Initialize variables
    R = (sqrt(5)-1)/2;  % Golden ratio
    d = R * (xU - xL);
    x1 = xL + d;
    x2 = xU - d;

    fxL = func(xL);
    fxU = func(xU);
    fx1 = func(x1);
    fx2 = func(x2);

    % Initialize history
    history.iteration = 0;
    history.xL = xL;
    history.xU = xU;
    history.x1 = x1;
    history.x2 = x2;
    history.fx1 = fx1;
    history.fx2 = fx2;
    history.error = inf;

    count = 0;
    error = inf;
    x_opt = xL;
    f_opt = fxL;

    fprintf('\n%-3s %-7s %-7s %-7s %-7s %-7s %-7s %-7s %-7s %-7s\n', ...
            'i', 'xL', 'f(xL)', 'x2', 'f(x2)', 'x1', 'f(x1)', 'xU', 'f(xU)', 'd', 'error');

    while error > err_limit && count < max_iter
        fprintf('%-3d %-7.4f %-7.4f %-7.4f %-7.4f %-7.4f %-7.4f %-7.4f %-7.4f %-7.4f %-7.4f\n', ...
                count, xL, fxL, x2, fx2, x1, fx1, xU, fxU, d, error);

        % Update the interval
        if fx1 > fx2
            % Maximum is in [x2, xU]
            x_opt = x1;
            f_opt = fx1;
            xL = x2;
            fxL = fx2;
            x2 = x1;
            fx2 = fx1;
            d = R * (xU - xL);
            x1 = xL + d;
            fx1 = func(x1);
        elseif fx1 < fx2
            % Maximum is in [xL, x1]
            x_opt = x2;
            f_opt = fx2;
            xU = x1;
            fxU = fx1;
            x1 = x2;
            fx1 = fx2;
            d = R * (xU - xL);
```

```
            x2 = xU - d;
            fx2 = func(x2);
        else
            % Maximum is at the midpoint
            xL = (x1 + x2)/2;
            xU = xL;
            x_opt = xL;
            f_opt = func(xL);
            break;
        end

        count = count + 1;
        error = (1 - R) * abs((xU - xL)/x_opt) * 100;

        % Store history
        history.iteration(count) = count;
        history.xL(count) = xL;
        history.xU(count) = xU;
        history.x1(count) = x1;
        history.x2(count) = x2;
        history.fx1(count) = fx1;
        history.fx2(count) = fx2;
        history.error(count) = error;
    end

    % Plot the function and optimum point
    x_vals = linspace(history.xL(1), history.xU(1), 1000);
    plot(x_vals, func(x_vals), 'b-', 'LineWidth', 2);
    hold on;
    plot(x_opt, f_opt, 'ro', 'MarkerSize', 10, 'MarkerFaceColor', 'r');
    xlabel('x');
    ylabel('f(x)');
    title('Golden-Section Search Optimization');
    grid on;
    hold off;
end
```

5.2 run_golden_section_example.m

```
% Example from pages 9-11: Maximize f(x) = 2*sin(x) - x^2/10
clc;
clear all;

% Define the function
func = @(x) 2*sin(x) - x.^2/10;

% Set parameters
xL = 0;
xU = 4;
err_limit = 0.000002; % 0.0002%
max_iter = 50;

% Run golden-section search
[x_opt, f_opt, history] = golden_section_search(func, xL, xU, err_limit,
    max_iter);

% Display results
fprintf('\nOptimal x: %.6f\n', x_opt);
fprintf('Optimal f(x): %.6f\n', f_opt);
```

5.3 beam_deflection.m

```
% Practice Problem 1: Beam deflection optimization
clc;
clear all;

% Given parameters
L = 600; % cm
E = 50000; % kN/cm^2
I = 30000; % cm^4
w0 = 2.5; % kN/cm

% Define the deflection function (we'll maximize the negative deflection to find minimum deflection)
deflection = @(x) w0/(120*E*I*L) * (-x.^5 + 2*L^2*x.^3 - L^4*x);

% Since we want to find maximum deflection, we'll maximize the deflection function
xL = 0;
xU = L;
err_limit = 1; % 1%
max_iter = 50;

% Run golden-section search
[x_opt, deflection_opt, history] = golden_section_search(deflection, xL, xU, err_limit, max_iter);

% Display results
fprintf('\nBeam Deflection Optimization:\n');
fprintf('Optimal x for maximum deflection: %.2f cm\n', x_opt);
fprintf('Maximum deflection: %.6f cm\n', deflection_opt);

% Plot deflection curve
x_vals = linspace(0, L, 1000);
deflection_vals = deflection(x_vals);
figure;
plot(x_vals, deflection_vals, 'b-', 'LineWidth', 2);
hold on;
plot(x_opt, deflection_opt, 'ro', 'MarkerSize', 10, 'MarkerFaceColor', 'r');
xlabel('Distance along beam (cm)');
ylabel('Deflection (cm)');
title('Beam Deflection');
grid on;
```

5.4 projectile_altitude.m

```
% Practice Problem 2: Projectile altitude optimization
clc;
clear all;

% Given parameters
g = 9.81; % m/s^2
z0 = 100; % m
v0 = 55; % m/s
m = 80; % kg
c = 15; % kg/s

% Define the altitude function
altitude = @(t) z0 + (m/c)*(v0 + m*g/c)*(1 - exp(-(c/m)*t)) - (m*g/c)*t;

% Set time bounds
tL = 0;
tU = 10; % s
err_limit = 1; % 1%
```

```
max_iter = 50;

% Run golden-section search
[t_opt, altitude_opt, history] = golden_section_search(altitude, tL, tU,
    err_limit, max_iter);

% Display results
fprintf('\nProjectile Altitude Optimization:\n');
fprintf('Optimal time for maximum altitude: %.4f s\n', t_opt);
fprintf('Maximum altitude: %.4f m\n', altitude_opt);

% Plot altitude curve
t_vals = linspace(0, 10, 1000);
altitude_vals = altitude(t_vals);
figure;
plot(t_vals, altitude_vals, 'b-', 'LineWidth', 2);
hold on;
plot(t_opt, altitude_opt, 'ro', 'MarkerSize', 10, 'MarkerFaceColor', 'r');
xlabel('Time (s)');
ylabel('Altitude (m)');
title('Projectile Altitude');
grid on;
```

6 ODE Solvers

6.1 euler_method.m

```
function [x, y, exact_soln, global_error, local_error] = euler_method(f,
    exact_eqn, f1, f2, f3, x0, y0, h, N)
    % Euler's method for solving ODE with error analysis

    % Initialize arrays
    x = zeros(1, N+1);
    y = zeros(1, N+1);
    exact_soln = zeros(1, N+1);
    global_error = zeros(1, N+1);
    local_error = zeros(1, N+1);

    % Set initial values
    x(1) = x0;
    y(1) = y0;
    exact_soln(1) = exact_eqn(x0);
    global_error(1) = 0;
    local_error(1) = 0;

    % Compute solutions using Euler's method
    for i = 1:N
        x(i+1) = x(i) + h;
        y(i+1) = y(i) + h * f(x(i), y(i));

        exact_soln(i+1) = exact_eqn(x(i+1));
        global_error(i+1) = abs(exact_soln(i+1) - y(i+1)) / abs(exact_soln(i+1)
    ) * 100;
        local_error(i+1) = abs(f1(x(i), y(i), h) + f2(x(i), y(i), h) + f3(x(i),
     y(i), h)) / abs(exact_soln(i+1)) * 100;
    end
end
```

6.2 runge_kutta4.m

```
function [x, y] = runge_kutta4(f, x0, y0, h, N)
    % Fourth-order Runge-Kutta method for solving ODE

    % Initialize arrays
    x = zeros(1, N+1);
    y = zeros(1, N+1);

    % Set initial values
    x(1) = x0;
    y(1) = y0;

    % Compute solutions using RK4 method
    for i = 1:N
        k1 = h * f(x(i), y(i));
        k2 = h * f(x(i) + h/2, y(i) + k1/2);
        k3 = h * f(x(i) + h/2, y(i) + k2/2);
        k4 = h * f(x(i) + h, y(i) + k3);

        y(i+1) = y(i) + (k1 + 2*k2 + 2*k3 + k4)/6;
        x(i+1) = x(i) + h;
    end
end
```

6.3 heun_method.m

```
function [X, Y] = heun_method(dydx, x0, y0, h, n)
    % Heun's method for solving ODE

    % Initialize arrays
    X = zeros(n+1, 1);
    Y = zeros(n+1, 1);

    % Set initial values
    X(1) = x0;
    Y(1) = y0;

    % Compute solutions using Heun's method
    for i = 1:n
        % Predictor step (Euler's method)
        y1 = Y(i) + h * dydx(X(i), Y(i));

        % Update X
        X(i+1) = X(i) + h;

        % Corrector step (Heun's method)
        y_pred = Y(i) + h/2 * (dydx(X(i), Y(i)) + dydx(X(i+1), y1));

        % Update Y
        Y(i+1) = y_pred;
    end
end
```

6.4 run_ode_examples.m

```
% Script to run all ODE examples from pages 13-17

clc;
clear all;

%% Example 1: Euler's method with error analysis (Page 13)
fprintf('=== Euler''s Method Example ===\n');
```

```

% Define the ODE and related functions
f = @(x, y) -2 * x.^3 + 12 * x.^2 - 20 * x + 8.5;
exact_eqn = @(x) -0.5 * x.^4 + 4 * x.^3 - 10 * x.^2 + 8.5 * x + 1;
f1 = @(x, y, h) (-6 * x.^2 + 24 * x - 20) * (h^2/2);
f2 = @(x, y, h) (-12 * x + 24) * (h^3/6);
f3 = @(x, y, h) (-12) * (h^4/24);

% Set initial conditions and parameters
x0 = 0;
y0 = 1;
h = 0.5;
N = 10;

% Run Euler's method
[x_euler, y_euler, exact, global_err, local_err] = euler_method(f, exact_eqn,
    f1, f2, f3, x0, y0, h, N);

% Display results
fprintf('x\t\tEuler\t\tExact\t\tGlobal Error\tLocal Error\n');
for i = 1:length(x_euler)
    fprintf('%.3f\t%.3f\t\t%.3f\t\t%.3f\t\t%.3f\n', ...
            x_euler(i), y_euler(i), exact(i), global_err(i), local_err(i));
end

% Plot results
figure;
subplot(2,1,1);
plot(x_euler, y_euler, 'b-o', x_euler, exact, 'r--');
xlabel('x');
ylabel('y');
title('Euler''s Method vs Exact Solution');
legend('Euler', 'Exact');
grid on;

subplot(2,1,2);
plot(x_euler, global_err, 'b-o', x_euler, local_err, 'r--');
xlabel('x');
ylabel('Error (%)');
title('Error Analysis');
legend('Global Error', 'Local Error');
grid on;

%% Example 2: Runge-Kutta 4th Order (Page 15)
fprintf('\n=== Runge-Kutta 4th Order Example ===\n');

% Run RK4 method
[x_rk4, y_rk4] = runge_kutta4(f, x0, y0, h, N);

% Display results
fprintf('x\t\tRK4\t\tExact\n');
for i = 1:length(x_rk4)
    fprintf('%.3f\t%.3f\t\t%.3f\n', x_rk4(i), y_rk4(i), exact(i));
end

% Plot results
figure;
plot(x_rk4, y_rk4, 'b-o', x_rk4, exact, 'r--');
xlabel('x');
ylabel('y');
title('Runge-Kutta 4th Order vs Exact Solution');
legend('RK4', 'Exact');
grid on;
```

```

%% Example 3: Heun's Method (Page 17)
fprintf('\n=== Heun''s Method Example ===\n');

% Define the ODE
dydx = @(x, y) 4*exp(0.8*x) - 0.5*y;

% Set initial conditions and parameters
x0 = 0;
y0 = 2;
h = 1;
n = 5;

% Run Heun's method
[X_heun, Y_heun] = heun_method(dydx, x0, y0, h, n);

% Display results
fprintf('x\t\tHeun\n');
for i = 1:length(X_heun)
    fprintf('%.3f\t%.3f\n', X_heun(i), Y_heun(i));
end

% Plot results
figure;
plot(X_heun, Y_heun, 'b-o');
xlabel('x');
ylabel('y');
title('Heun''s Method Solution');
grid on;
```

6.5 compare_ode_methods.m

```
% Script to compare different ODE solving methods

clc;
clear all;

% Define the ODE
f = @(x, y) -2 * x.^3 + 12 * x.^2 - 20 * x + 8.5;
exact_eqn = @(x) -0.5 * x.^4 + 4 * x.^3 - 10 * x.^2 + 8.5 * x + 1;

% Set initial conditions and parameters
x0 = 0;
y0 = 1;
h = 0.5;
N = 10;

% Run all methods
[x_euler, y_euler] = euler_method(f, exact_eqn, @(x,y,h)0, @(x,y,h)0, @(x,y,h)
    0, x0, y0, h, N);
[x_rk4, y_rk4] = runge_kutta4(f, x0, y0, h, N);
exact = exact_eqn(x_euler);

% Calculate errors
error_euler = abs(y_euler - exact);
error_rk4 = abs(y_rk4 - exact);

% Display comparison
fprintf('Comparison of ODE Methods\n');
fprintf('x\t\tEuler\t\tRK4\t\tExact\t\tEuler Error\tRK4 Error\n');
for i = 1:length(x_euler)
    fprintf('%.3f\t%.3f\t\t%.3f\t\t%.3f\t\t%.3f\t\t%.3f\n', ...
```

```
            x_euler(i), y_euler(i), y_rk4(i), exact(i), error_euler(i), error_rk4(i));
end

% Plot comparison
figure;
subplot(2,1,1);
plot(x_euler, y_euler, 'b-o', x_rk4, y_rk4, 'g-s', x_euler, exact, 'r--');
xlabel('x');
ylabel('y');
title('Comparison of ODE Methods');
legend('Euler', 'RK4', 'Exact');
grid on;

subplot(2,1,2);
semilogy(x_euler, error_euler, 'b-o', x_rk4, error_rk4, 'g-s');
xlabel('x');
ylabel('Error');
title('Error Comparison');
legend('Euler Error', 'RK4 Error');
grid on;
```

7 Numerical Integration

7.1 trapezoidal_rule.m

```
function [I_approx, I_exact, true_error] = trapezoidal_rule(f, a, b, n_values)
    % Initialize arrays to store results
    I_approx = zeros(size(n_values));
    I_exact = zeros(size(n_values));
    true_error = zeros(size(n_values));

    % Compute exact integral using symbolic math
    syms x_sym;
    I_exact_val = double(int(f(x_sym), a, b));

    for i = 1:length(n_values)
        n = n_values(i);

        % Compute the width of each subinterval
        h = (b - a) / n;

        % Compute the values of the function at the endpoints
        x = linspace(a, b, n+1);
        y = f(x);

        % Compute the approximate integral using trapezoidal rule
        I_approx(i) = h/2 * (y(1) + 2*sum(y(2:n)) + y(n+1));

        % Compute the error
        true_error(i) = abs(I_exact_val - I_approx(i)) / abs(I_exact_val) * 100;
    end

    % Set all exact values to the same value
    I_exact(:) = I_exact_val;
end
```

7.2 simpsons_13_rule.m

```
function [I_approx, I_exact, rel_error] = simpsons_13_rule(f, a, b, n_values)
    % Initialize arrays to store results
    I_approx = zeros(size(n_values));
    I_exact = zeros(size(n_values));
    rel_error = zeros(size(n_values));

    % Compute exact integral using symbolic math
    syms x_sym;
    I_exact_val = double(int(f(x_sym), a, b));

    for i = 1:length(n_values)
        n = n_values(i);

        % Simpson's 1/3 rule requires even number of subintervals
        if mod(n, 2) ~= 0
            error('Simpson''s 1/3 rule requires an even number of subintervals.');
        end

        % Compute the width of each subinterval
        h = (b - a) / n;

        % Compute the values of the function at the endpoints
        x = linspace(a, b, n+1);
        y = f(x);

        % Compute the approximate integral using Simpson's 1/3 rule
        I_approx(i) = h/3 * (y(1) + 4*sum(y(2:2:n)) + 2*sum(y(3:2:n-1)) + y(n+1));

        % Compute the relative error
        rel_error(i) = abs(I_exact_val - I_approx(i)) / abs(I_exact_val) * 100;
    end

    % Set all exact values to the same value
    I_exact(:) = I_exact_val;
end
```

7.3 simpsons_38_rule.m

```
function [I_approx, I_exact, true_error] = simpsons_38_rule(f, a, b, n_values)
    % Initialize arrays to store results
    I_approx = zeros(size(n_values));
    I_exact = zeros(size(n_values));
    true_error = zeros(size(n_values));

    % Compute exact integral using symbolic math
    syms x_sym;
    I_exact_val = double(int(f(x_sym), a, b));

    for i = 1:length(n_values)
        n = n_values(i);

        % Simpson's 3/8 rule requires multiple of 3 subintervals
        if mod(n, 3) ~= 0
            error('Simpson''s 3/8 rule requires a multiple of 3 subintervals.');
        end

        % Compute the width of each subinterval
        h = (b - a) / n;

```

```
        % Compute the values of the function at the endpoints
        x = linspace(a, b, n+1);
        y = f(x);

        % Compute the approximate integral using Simpson's 3/8 rule
        I_approx(i) = 3*h/8 * (y(1) + 3*sum(y(2:3:n-1) + y(3:3:n)) + 2*sum(y(4:3:n-2)) + y(n+1));

        % Compute the error
        true_error(i) = abs(I_exact_val - I_approx(i)) / abs(I_exact_val) * 100;
    end

    % Set all exact values to the same value
    I_exact(:) = I_exact_val;
end
```

7.4 run_integration_examples.m

```
% Script to run all integration examples from pages 18-21

clc;
clear all;

% Define the function to be integrated
f = @(x) 0.2 + 25*x - 200*x.^2 + 675*x.^3 - 900*x.^4 + 400*x.^5;

% Define the limits of integration
a = 0;
b = 0.8;

% Define the values of n to try
n_values = 1:10;

%% Trapezoidal Rule (Page 18)
fprintf('=== Trapezoidal Rule ===\n');
[I_trap, I_exact_trap, error_trap] = trapezoidal_rule(f, a, b, n_values);

fprintf('n\tApproximate\tExact\t\tError (%%)\n');
for i = 1:length(n_values)
    fprintf('%d\t%.6f\t%.6f\t%.6f\n', n_values(i), I_trap(i), I_exact_trap(i), error_trap(i));
end

%% Simpson's 1/3 Rule (Pages 19-20)
fprintf('\n=== Simpson''s 1/3 Rule ===\n');
% Only use even values of n for Simpson's 1/3 rule
n_values_simp13 = n_values(mod(n_values, 2) == 0);
[I_simp13, I_exact_simp13, error_simp13] = simpsons_13_rule(f, a, b, n_values_simp13);

fprintf('n\tApproximate\tExact\t\tError (%%)\n');
for i = 1:length(n_values_simp13)
    fprintf('%d\t%.6f\t%.6f\t%.6f\n', n_values_simp13(i), I_simp13(i), I_exact_simp13(i), error_simp13(i));
end

%% Simpson's 3/8 Rule (Page 21)
fprintf('\n=== Simpson''s 3/8 Rule ===\n');
% Only use multiples of 3 for Simpson's 3/8 rule
n_values_simp38 = [3, 6, 9];
```

```
[I_simp38, I_exact_simp38, error_simp38] = simpsons_38_rule(f, a, b,
    n_values_simp38);

fprintf('n\tApproximate\tExact\t\tError (%%)\n');
for i = 1:length(n_values_simp38)
    fprintf('%d\t%.6f\t%.6f\t%.6f\n', n_values_simp38(i), I_simp38(i),
    I_exact_simp38(i), error_simp38(i));
end

%% Plot the function
figure;
x_vals = linspace(a, b, 1000);
y_vals = f(x_vals);
plot(x_vals, y_vals, 'b-', 'LineWidth', 2);
xlabel('x');
ylabel('f(x)');
title('Function: f(x) = 0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5');
grid on;

%% Compare methods
figure;
semilogy(n_values, error_trap, 'b-o', 'LineWidth', 2);
hold on;
semilogy(n_values_simp13, error_simp13, 'r-*', 'LineWidth', 2);
semilogy(n_values_simp38, error_simp38, 'g-s', 'LineWidth', 2);
xlabel('Number of subintervals (n)');
ylabel('Relative Error (%)');
title('Comparison of Integration Methods');
legend('Trapezoidal Rule', 'Simpson''s 1/3 Rule', 'Simpson''s 3/8 Rule');
grid on;
```